ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



NỀN TẢNG LẬP TRÌNH CHO PHÂN TÍCH
VÀ TRỰC QUAN DỮ LIỆU
Mã môn học: CO5177

BÁO CÁO IMAGE

CNN vs Transformer trên CIFAR-100

ResNet18 và Swin-T

Học kỳ 261 · Năm học 2026–2027

Giảng viên: Lê Thành Sách

Thành viên: Huỳnh Thành Đạt · 2570161
Lương Minh Duy · 2570083

Thành phố Hồ Chí Minh, tháng 10 năm 2026

Mục lục

1 Bài toán và dữ liệu **1**
1.1 Mục tiêu và phạm vi . . . 1
1.2 CIFAR-100 và tình trạng license . . . 1
1.3 Split và tiền xử lý . . . 1
1.4 Đối chiếu với yêu cầu Image . . . 2
1.5 Khám phá dữ liệu và kiểm tra ảnh trùng . . . 2

2 Kiến trúc và chế độ huấn luyện **4**
2.1 ResNet18: đối chứng CNN . . . 4
2.2 Swin-T: đối chứng Transformer . . . 4
2.3 Ma trận đối chứng . . . 5

3 Thiết lập thực nghiệm và phép đo **6**
3.1 Thiết lập huấn luyện . . . 6
3.2 Chọn checkpoint và mở test . . . 6
3.3 Chi phí và môi trường . . . 7
3.4 Checkpoint và thông tin tái lập . . . 7

4 Kết quả thực nghiệm và thảo luận **8**
4.1 So sánh các chế độ huấn luyện . . . 8
4.2 Thời gian, bộ nhớ và số tham số . . . 8
4.3 Thảo luận kết quả . . . 9
4.4 Đường cong và lỗi phân loại . . . 10

5 Kết luận và giới hạn **15**
5.1 Kết luận . . . 15
5.2 Tổ chức mã nguồn và tái lập . . . 15
5.3 Giới hạn . . . 15

Tài liệu tham khảo **16**

Chương 1

Bài toán và dữ liệu

1.1 Mục tiêu và phạm vi

Phần Image so sánh CNN và Transformer trên CIFAR-100, tập trung vào ảnh hưởng của khởi tạo pretrained và mức độ đóng băng backbone. Hai kiến trúc được chọn là ResNet18 và Swin-T của torchvision. Mỗi kiến trúc có bốn chế độ huấn luyện, chạy với ba seed 42, 43 và 44, tổng cộng 24 lượt chạy.

So sánh chính của bài Image gồm sáu cấu hình pretrained: head-only, partial và full fine-tuning của mỗi kiến trúc, tương ứng 18 lượt chạy. Hai cấu hình scratch với sáu lượt chạy còn lại là đối chứng bổ sung. Accuracy, macro-F1, thời gian và bộ nhớ được dùng để đánh giá hiệu quả cùng chi phí của từng chế độ.

1.2 CIFAR-100 và tình trạng license

CIFAR-100 gồm 60.000 ảnh màu 32×32, 100 nhãn chi tiết (fine classes) và 20 nhóm coarse classes. Bài này dự đoán toàn bộ 100 fine classes; không chọn subset ảnh hoặc lớp. Mỗi lớp có 500 ảnh official train và 100 ảnh official test [2]. Ảnh độ phân giải thấp có thể làm những lớp gần nhau khó phân biệt.

Nguồn chính là trang CIFAR của Toronto, của Alex Krizhevsky, Vinod Nair và Geoffrey Hinton. Trang này không nêu license tường minh; thẻ dataset chính thức `uoft-cs/cifar100` trên Hugging Face ghi *license: unknown* [2, 3]. Vì vậy, tình trạng license của dữ liệu được ghi là chưa rõ. Ảnh gốc được tải từ Toronto để thực hiện thí nghiệm và lưu ngoài repository.

1.3 Split và tiền xử lý

Từ 50.000 ảnh train gốc, shuffle indices riêng từng lớp bằng NumPy `default_rng(2026)`. Lấy 50 ảnh/lớp làm validation, 450 ảnh/lớp làm train: 45.000 train và 5.000 validation. Indices được sắp tăng sau khi chọn và lưu cùng hash. Official test 10.000 ảnh nằm riêng, không dùng cập nhật tham số hoặc chọn checkpoint. Train seed 42/43/44 không thay split seed 2026.

Train dùng RandomCrop 32×32 với padding 4, horizontal flip xác suất 0,5, sau đó bilinear resize 96×96 có antialias. Val/test chỉ resize, không crop ngẫu nhiên hoặc flip. Cả hai họ và mọi chế độ dùng ImageNet mean $(0{,}485, 0{,}456, 0{,}406)$, std $(0{,}229, 0{,}224, 0{,}225)$ sau ToTensor. Không dùng trực tiếp recipe `weights.transforms()` của pretrained.

Augmentation dùng seed theo train seed, epoch và chỉ số ảnh để giữ phép biến đổi nhất quán

khi resume.

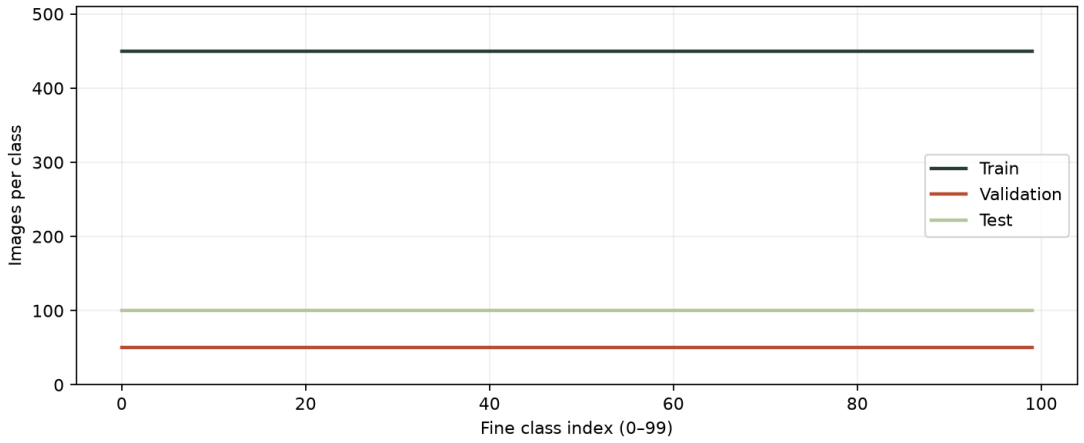
1.4 Đối chiếu với yêu cầu Image

CIFAR-100 có đủ số ảnh và số lớp cho bài Image, đồng thời không thuộc ba tập MNIST, Fashion-MNIST và CIFAR-10 bị loại khỏi bài chính. Sáu cấu hình head/partial/full sử dụng pretrained theo yêu cầu khi dùng deep learning; hai cấu hình scratch chỉ là đối chứng bổ sung. Ảnh gốc có cạnh 32px, thấp hơn mức 128px được khuyến nghị trong brief. Kích thước 96px được giữ để đối chiếu cùng protocol và chi phí; phép resize không bổ sung thông tin.

Các phép đo trong báo cáo được dùng lại từ A1 CO5085 tại commit `1250e56`, không phải một lượt train độc lập của CO5177. Code được chuyển sang `src/image/`; manifest ghi nguồn, hash và các thay đổi import/đường dẫn. Notebook thực hiện kiểm tra dữ liệu và tính lại chỉ số từ predictions. Không thay protocol hoặc chọn checkpoint mới sau khi đọc test.

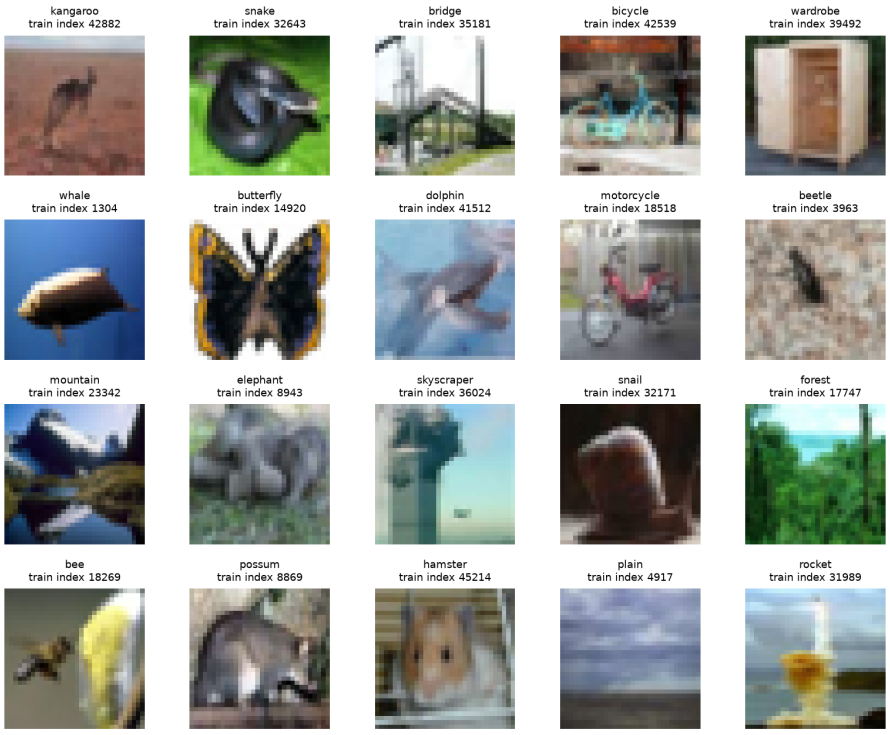
1.5 Khám phá dữ liệu và kiểm tra ảnh trùng

Phần khám phá dùng mirror `uoft-cs/cifar100` tại revision cố định. Hash toàn bộ pixel và nhãn theo thứ tự của cả 50.000 ảnh development và 10.000 ảnh test khớp với dữ liệu Toronto trong metadata của các lượt chạy. Tất cả ảnh đều là RGB 32×32. Mỗi lớp có 450 ảnh train, 50 ảnh validation và 100 ảnh test, nên không có mất cân bằng theo số lượng fine classes. Tuy nhiên, cân bằng số mẫu không bảo đảm các lớp có độ khó tương đương.



Hình 1.1: Số ảnh từng fine class sau split. Các lớp có cùng số ảnh trong mỗi split.

Kiểm tra SHA256 trên RGB bytes phát hiện 14 ảnh lặp nội dung trong tập development 50.000 ảnh. Trong split đã đo có 2 nội dung ảnh trùng giữa train và validation, 10 nội dung trùng giữa train và test, không có trùng validation–test. Test có 9.998 nội dung ảnh duy nhất trên 10.000 bản ghi. Đây là kiểm tra trùng pixel hoàn toàn, chưa đánh giá near duplicates. Vì vậy, các split tách rời về chỉ số nhưng không hoàn toàn tách rời về nội dung ảnh.



Hình 1.2: Hai mươi ảnh train chọn bằng seed 2026, kèm nhãn và chỉ số gốc. Ảnh được phóng lớn để quan sát; chi tiết phân biệt một số lớp vẫn hạn chế.

Bộ split và checkpoint đã công bố được giữ nguyên. Kiểm tra độ nhạy bổ sung loại 10 ảnh test trùng train, giữ 9990 prediction của cùng các checkpoint. ResNet18 full đạt 79.009%, so với 78.970% ban đầu; Swin-T full đạt 86.677%, so với 86.630% ban đầu. Chênh lệch nhỏ trong kiểm tra này không chứng minh quy trình hoàn toàn không có leakage: hai nội dung trùng train–validation vẫn có thể ảnh hưởng chọn checkpoint. Không gọi đây là kết quả train lại sau loại trùng.

Một protocol tiếp theo cần nhóm các ảnh trùng trước khi chia dữ liệu và đo lại toàn bộ đối chứng. Không thay split sau khi xem test rồi gộp phép đo mới với bảng cũ. Các chỉ số và danh sách ảnh trùng nằm trong `analysis/dataset.json`, `overlap.json` và `sensitivity.json`.

Chương 2

Kiến trúc và chế độ huấn luyện

2.1 ResNet18: đối chứng CNN

Giữ kiến trúc ResNet18 chuẩn torchvision, gồm stem convolution 7×7, max pooling và bốn residual stages. Một residual block cập nhật

$$h_{l+1} = \text{ReLU}\left(F_l(h_l) + S_l(h_l)\right), \tag{2.1}$$

trong đó S_l là identity hoặc projection khi đổi số kênh/stride. Kernel convolution chia sẻ trọng số theo vị trí và khai thác quan hệ cục bộ. Sau adaptive average pooling, head Linear được thay để dự đoán 100 lớp; không thay stem thành biến thể ResNet chuyên cho CIFAR.

Bản pretrained dùng `ResNet18_Weights.IMAGENET1K_V1`. Ba chế độ là chỉ học `fc`; học `layer4` và `fc`; học toàn bộ. Bản scratch khởi tạo mới và học toàn bộ. BatchNorm trong phần frozen ở eval, nên cả tham số lẫn running mean/variance và bộ đếm batch được giữ nguyên.

2.2 Swin-T: đối chứng Transformer

Swin-T dùng patch embedding stride4, các stage Transformer theo cấp độ và patch merging. Attention trên cửa sổ có dạng

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}} + B + M\right)V, \tag{2.2}$$

trong đó B là relative position bias, M là mask của shifted windows. Dịch cửa sổ giúp trao đổi thông tin giữa cửa sổ qua các block; việc giới hạn attention trong cửa sổ khác với global attention của ViT cổ điển.

Swin-T giữ kiến trúc chuẩn torchvision, với `head` được thay bằng Linear 100 lớp. Đầu vào 96×96 tạo lưới patch đầu 24×24; các stage muộn có lưới nhỏ hơn. Kích thước cửa sổ mặc định là 7, với padding nội bộ ở những stage có kích thước không chia hết cho cửa sổ.

Bản pretrained dùng `Swin_T_Weights.IMAGENET1K_V1`. Chế độ partial chỉ học stage cuối `features.7`, `norm` cuối và `head`; patch merging đứng trước stage cuối vẫn frozen. Head-only chỉ học head. Frozen stages được giữ ở eval, bao gồm stochastic depth, để đặc trưng của phần đóng băng ổn định trong quá trình học head. Full và scratch cập nhật toàn bộ mô hình.

2.3 Ma trận đối chứng

Chế độ	Khởi tạo backbone	Phần được cập nhật
Scratch	Ngẫu nhiên	Toàn bộ
Head-only	ImageNet pretrained	Head100
Partial	ImageNet pretrained	Stage cuối và head; Swin thêm norm
Full	ImageNet pretrained	Toàn bộ

Bảng 2.1: Bốn chế độ cho mỗi họ kiến trúc, cùng split và ba seed.

Scratch cập nhật toàn bộ mô hình từ khởi tạo ngẫu nhiên. Số tham số tổng và trainable được đếm trực tiếp từ từng mô hình và lưu trong metadata. ResNet18 và Swin-T khác nhau về kiến trúc, capacity và recipe pretrained; các kết quả phản ánh tổng thể những khác biệt này trong cùng thiết lập.

Chương 3

Thiết lập thực nghiệm và phép đo

3.1 Thiết lập huấn luyện

Mỗi lượt huấn luyện có 20 epoch, batch 128 (giữ batch cuối), vòng lặp huấn luyện trực tiếp, AdamW với weight decay 10^{-4}, $\beta = (0{,}9, 0{,}999)$, epsilon 10^{-8}. Learning rate ban đầu là 10^{-3} cho scratch/head và 10^{-4} cho pretrained partial/full. Cosine scheduler bước một lần sau epoch:

$$\eta_e = \frac{\eta_0}{2}\left[1 + \cos\left(\frac{\pi e}{20}\right)\right], \tag{3.1}$$

với $e = 0$ trước epoch đầu. Loss cross-entropy không label smoothing:

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp z_{i,y_i}}{\sum_{c=1}^{100} \exp z_{i,c}}. \tag{3.2}$$

Clip global gradient norm ở 1 trước optimizer.step. BF16 autocast áp dụng cho tất cả chế độ; tham số và Adam moments giữ FP32, không GradScaler, TF32 tắt và không dùng compile. Các phép đo được thiết lập trên RTX 5090 với 8 workers và 4 CPU threads.

Ngân sách 20 epoch được giữ cố định giữa các chế độ. Learning rate của head/scratch và partial/full được đặt khác nhau để phù hợp với mức độ cập nhật backbone. Kết quả vì vậy được phân tích theo từng thiết lập đầy đủ; mức hội tụ còn được đối chiếu qua đường cong train/validation.

3.2 Chọn checkpoint và mở test

Best chọn bằng val accuracy lớn nhất; hòa accuracy thì lấy CE nhỏ nhất, hòa cả hai giữ epoch trước. Chỉ val được dùng ra quyết định trong train. Test được đánh giá ở một bước riêng sau khi hoàn tất các lượt huấn luyện. Các cấu hình, learning rate và seed được cố định trước bước này.

Accuracy và macroF1 đều tính trên 100 lớp. Với mỗi lớp c,

$$F1_c = \frac{2TP_c}{2TP_c + FP_c + FN_c}, \qquad F1_{macro} = \frac{1}{100}\sum_{c=1}^{100} F1_c. \tag{3.3}$$

Trường hợp mẫu số bằng0 có F1 bằng0. Confusion matrix, support, precision, recall, F1 theo lớp và từng prediction được lưu để kiểm tra. Gallery lấy tối đa 8 lỗi confidence cao nhất của seed 42.

3.3 Chi phí và môi trường

Training time bao gồm data loading/augmentation và train pass, có CUDA synchronize trước/sau. Epoch time thêm validation. Model construction, checkpoint và report export nằm ngoài hai khoảng đo; không gọi training time là tổng wall time của job. Peak GPU memory là peak allocated trong train và validation của mỗi epoch, không phải reserved memory hoặc VRAM toàn hệ thống.

Metadata lưu GPU, torch/torchvision, CUDA/cuDNN, workers, CPU threads và chế độ deterministic. Các phép đo thời gian của cùng cấu hình được gộp khi những thông số môi trường này thống nhất.

3.4 Checkpoint và thông tin tái lập

Run lưu `last.pt` với model, optimizer, scheduler, RNG Python/NumPy/ PyTorch/CUDA, loader generators và history; `best.pt` chỉ lưu model và selection. Không giữ best optimizer hay checkpoint mỗi epoch. Atomic writes và khóa run bảo vệ các tệp kết quả. Resume từ epoch hoàn tất gần nhất, không giữa batch; best write bị ngắt có thể khôi phục từ last khi last là best epoch.

Config/source/data/split/environment hashes bảo vệ resume và test. Frozen parameters/buffers được fingerprint kiểm tra sau epoch. Git commit/dirty được ghi riêng để đối chiếu phiên bản mã nguồn. Checkpoint được lưu ngoài Git; JSON/CSV và checksums được giữ trong generated/evidence. Sai khác GPU hoặc kernel có thể ảnh hưởng khả năng tái lập từng bit.

Chương 4

Kết quả thực nghiệm và thảo luận

4.1 So sánh các chế độ huấn luyện

Kiến trúc	Chế độ	Val (%)	Test (%)	Macro-F1
ResNet18	Scratch	63.83 ± 0.51	64.35 ± 0.11	0.6422 ± 0.0012
ResNet18	Head-only	61.89 ± 0.24	60.95 ± 0.29	0.6081 ± 0.0030
ResNet18	Partial	74.50 ± 0.31	73.96 ± 0.12	0.7396 ± 0.0011
ResNet18	Full	79.29 ± 0.10	78.97 ± 0.04	0.7898 ± 0.0006
Swin-T	Scratch	2.81 ± 1.40	2.76 ± 1.26	0.0093 ± 0.0069
Swin-T	Head-only	72.45 ± 0.10	71.67 ± 0.09	0.7154 ± 0.0009
Swin-T	Partial	78.99 ± 0.31	78.25 ± 0.22	0.7818 ± 0.0021
Swin-T	Full	86.89 ± 0.02	86.63 ± 0.22	0.8663 ± 0.0020

Bảng 4.1: Accuracy và macro-F1 của tám cấu hình, trung bình và SD mẫu qua ba seed. Accuracy theo phần trăm, F1 theo thang 0–1.

Kết quả được tổng hợp bằng trung bình và độ lệch chuẩn mẫu qua các seed. Danh sách lượt chạy và checkpoint tương ứng được lưu trong manifest.

4.2 Thời gian, bộ nhớ và số tham số

Kiến trúc	Chế độ	Tổng (M)	Học (M)	Train (s)	VRAM (MiB)
ResNet18	Scratch	11.228	11.228	253.4 ± 1.2	522.5
ResNet18	Head-only	11.228	0.051	155.9 ± 2.9	252.1
ResNet18	Partial	11.228	8.445	194.1 ± 0.2	347.3
ResNet18	Full	11.228	11.228	257.6 ± 3.2	522.4
Swin-T	Scratch	27.596	27.596	727.1 ± 4.8	2456.9
Swin-T	Head-only	27.596	0.077	265.3 ± 2.1	502.2
Swin-T	Partial	27.596	14.262	347.5 ± 0.4	663.6
Swin-T	Full	27.596	27.596	724.4 ± 1.7	2456.9

Bảng 4.2: Tổng tham số và tham số được học theo triệu (M); thời gian train đủ 20 epoch; peak allocated memory trung bình theo MiB. Phạm vi phép đo ở Mục 3.3.

4.3 Thảo luận kết quả

Kết quả gồm 24 lượt chạy, với ba seed 42, 43 và 44 cho mỗi cấu hình. Chênh lệch accuracy được tính theo cặp cùng seed; các khoảng $\pm$ là độ lệch chuẩn mẫu, không phải khoảng tin cậy.

4.3.1 Khởi tạo và khả năng thích nghi biểu diễn

ResNet18 head-only so với scratch có chênh lệch -3.40 $\pm$ 0.33 điểm phần trăm. Backbone đóng băng chỉ cho phép head tuyến tính điều chỉnh ranh giới phân loại trên đặc trưng có sẵn. Kết quả cho thấy đặc trưng ImageNet cố định chưa đủ phù hợp với thiết lập CIFAR-100 này. Mở stage cuối tăng +9.61 $\pm$ 0.09 điểm phần trăm so với scratch; mở toàn bộ tăng +14.62 $\pm$ 0.11 điểm phần trăm. Khả năng cập nhật biểu diễn đóng vai trò đáng kể đối với ResNet18.

Swin-T scratch chỉ đạt $2.76 \pm 1.26\%$ trên test. Ở epoch 20, seed 43 và 44 có validation accuracy 1%, train accuracy xấp xỉ 1% và CE gần $\ln(100) \simeq 4{,}605$. Cả train lẫn validation đều thấp, cho thấy mô hình học rất ít trong thiết lập này. Seed 42 cải thiện hơn nhưng vẫn xa các bản pretrained. Ngân sách 20 epoch, learning rate 10^{-3} và lịch cosine không warm-up chưa tạo được một đối chứng scratch hội tụ tốt. Thực nghiệm không tách riêng nguyên nhân của từng yếu tố và không khảo sát thêm siêu tham số sau khi mở test.

Swin-T head-only đạt $71.67 \pm 0.09\%$, partial đạt $78.25 \pm 0.22\%$, và full đạt $86.63 \pm 0.22\%$. Full cao hơn scratch +83.87 $\pm$ 1.40 điểm phần trăm; mức chênh lệch này phản ánh cả lợi ích của pretrained lẫn đối chứng scratch chưa học tốt. Kết quả không đủ để kết luận Transformer luôn phụ thuộc pretrained hoặc luôn vượt CNN trong những ngân sách khác.

4.3.2 Accuracy và chi phí huấn luyện

Khi cùng fine-tuning toàn bộ, Swin-T cao hơn ResNet18 +7.66 $\pm$ 0.18 điểm phần trăm, với macro-F1 trung bình 0.8663 so với 0.7898. Thời gian train tăng 2.81 lần và peak allocated memory tăng 4.70 lần. Đây là sự đánh đổi giữa accuracy và chi phí trong cùng ngân sách epoch; hai kiến trúc vẫn khác số tham số và quá trình pretraining.

Head-only giảm peak allocated memory so với full từ 522.4 xuống 252.1 MiB ở ResNet18, và từ 2456.9 xuống 502.2 MiB ở Swin-T. Đóng băng backbone giảm bộ nhớ gradient và trạng thái optimizer, nhưng vẫn cần bước forward qua toàn bộ backbone.

4.3.3 Đường cong học và lỗi phân loại

Ở epoch 20, train/validation accuracy của các bản full có khoảng cách rõ: ResNet18 99.07%/79.18%; Swin-T 98.68%/86.75%. Train cải thiện nhanh hơn validation, trong khi validation CE còn cao hơn train CE; đây là dấu hiệu overfit. Điểm test trong bảng lấy từ checkpoint tốt nhất theo validation, không nhất thiết là epoch 20.

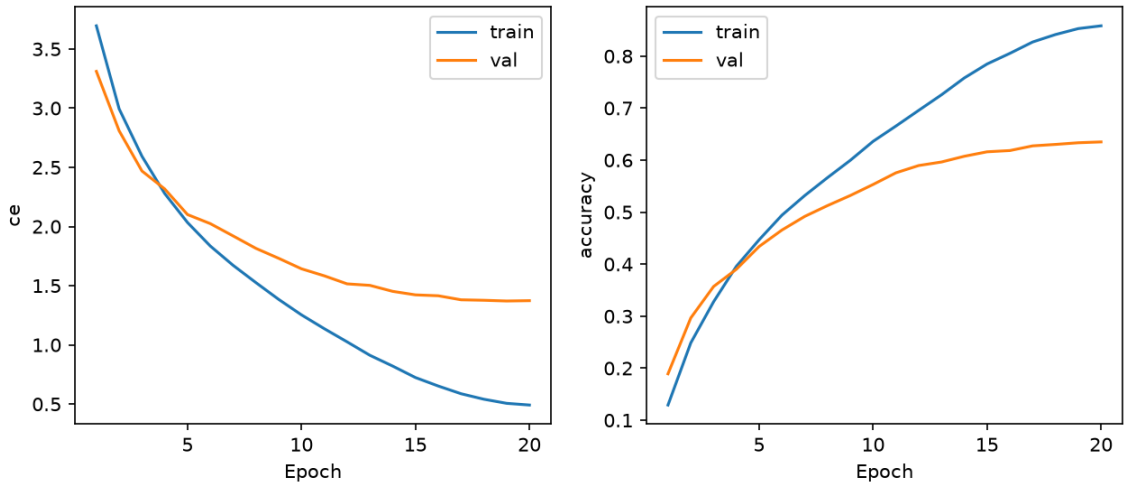
ResNet18 full ở seed 42: lỗi nhiều nhất theo một chiều là oak_tree thành maple_tree, 23 ảnh. Lớp oak_tree có 100 ảnh trong tập test.

Swin-T full ở seed 42: lỗi nhiều nhất theo một chiều là oak_tree thành maple_tree, 22 ảnh. Lớp oak_tree có 100 ảnh trong tập test.

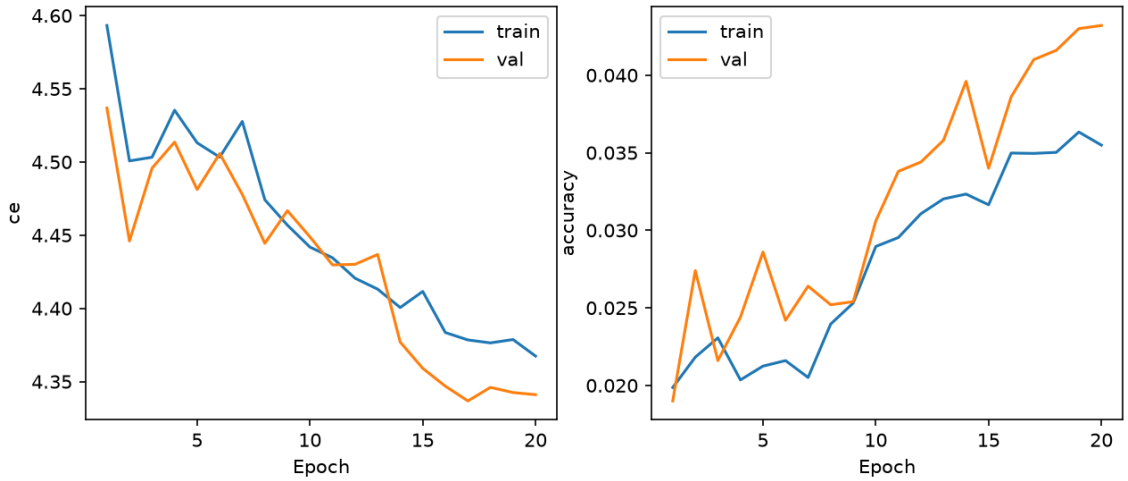
Hai mô hình cùng nhầm oak_tree với maple_tree, gợi ý khó khăn khi phân biệt các lớp cây có hình dáng và kết cấu gần nhau trong ảnh 32×32. Đây là cách diễn giải từ ma trận lỗi và các ví dụ quan sát, không phải phép đo riêng về mức tương đồng thị giác. Những lỗi confidence cao cho thấy accuracy cao vẫn đi kèm một số dự đoán sai rất tự tin; calibration chưa được đánh giá riêng.

4.4 Đường cong và lỗi phân loại

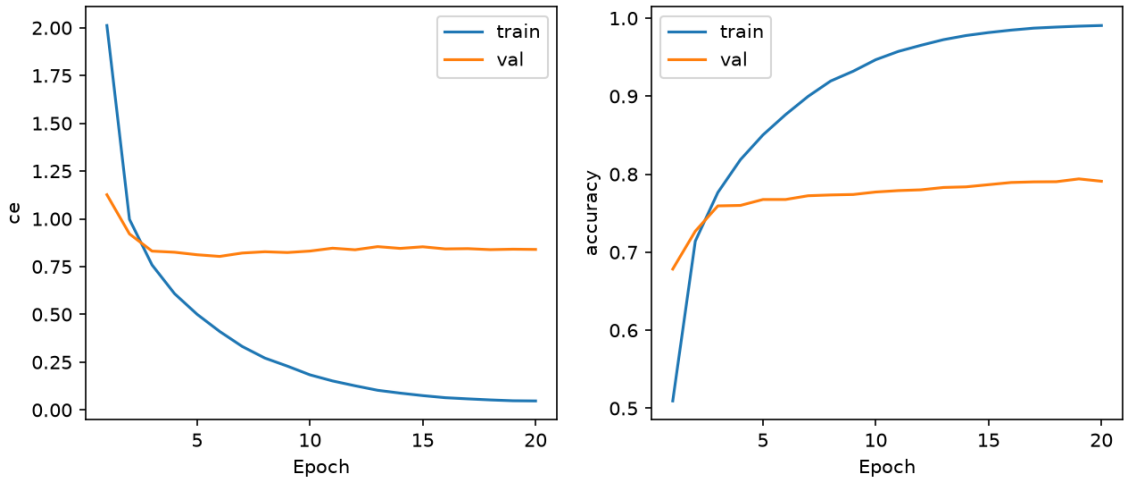
Các hình dùng seed 42 đã định trước, không chọn seed theo kết quả test. Ma trận lỗi dùng chỉ số lớp 0–99; danh sách tên lớp và số đếm đầy đủ nằm trong các tệp CSV kèm báo cáo. Notebook trình bày đường cong của cả sáu cấu hình pretrained. Các hình dưới đối chiếu scratch và full fine-tuning; các tệp hình được dùng lại cùng lịch sử 20 epoch của phép đo gốc, với SHA256 trong manifest. Ở các bản full, accuracy train tiếp tục tăng trong khi validation cải thiện chậm hơn. Khoảng cách này phù hợp với dấu hiệu overfit đã nêu ở phần thảo luận.



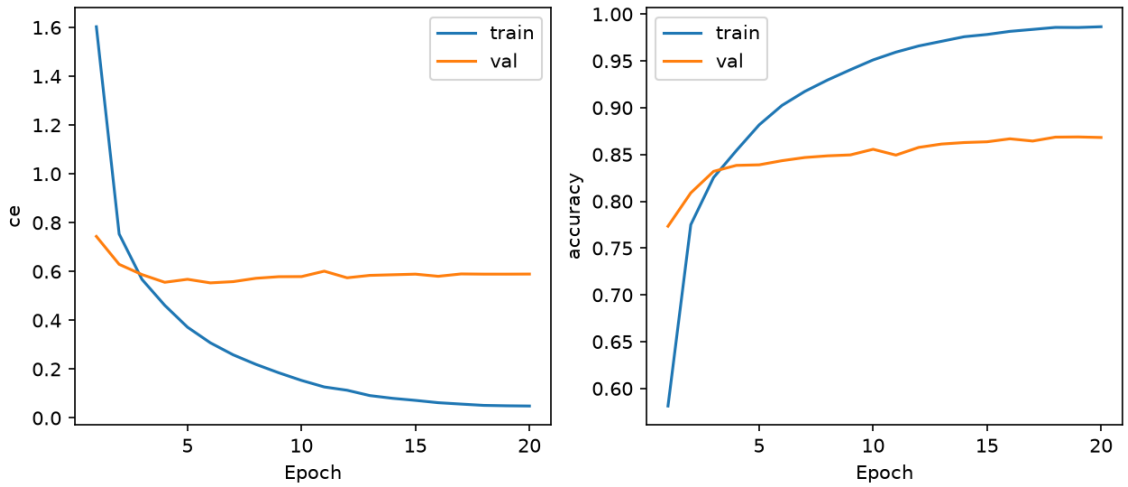
Hình 4.1: ResNet18 scratch: train/val CE và accuracy ở seed 42.



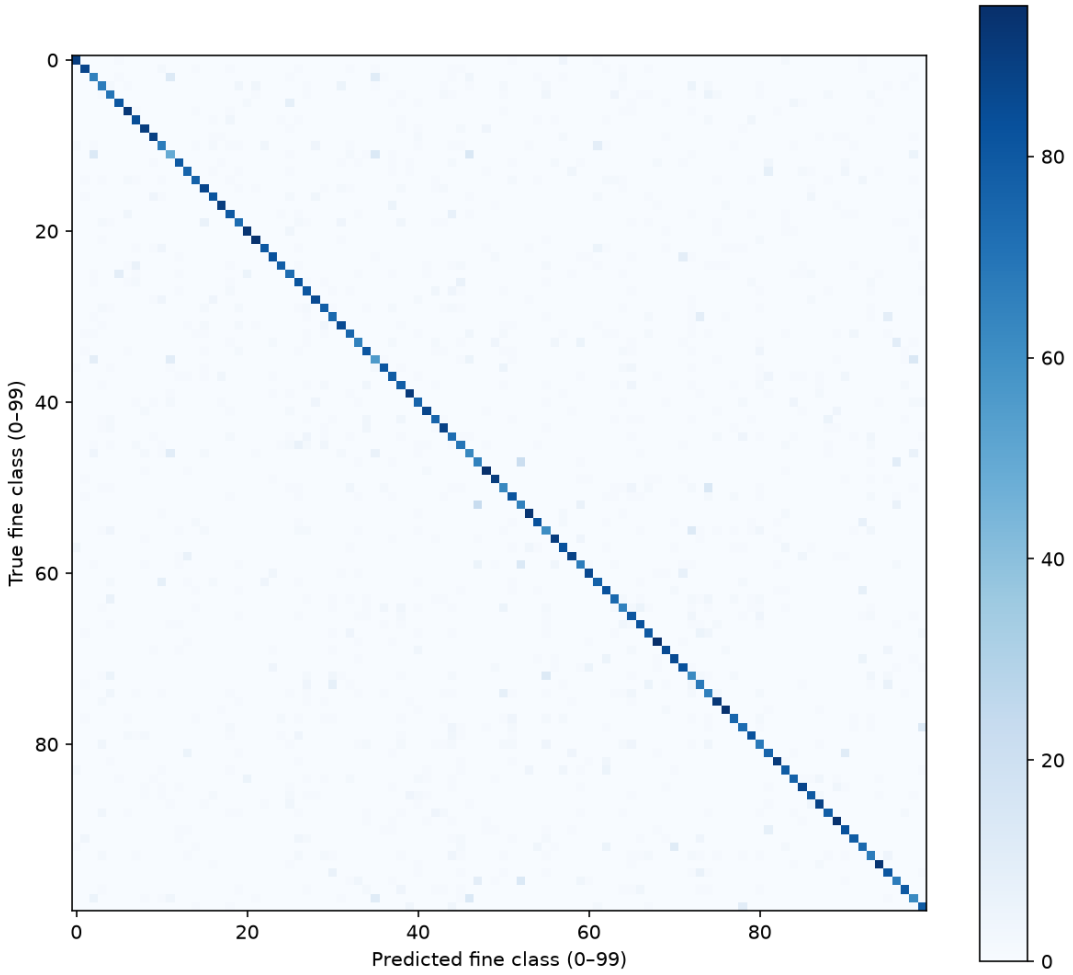
Hình 4.2: Swin-T scratch: train/val CE và accuracy ở seed 42.



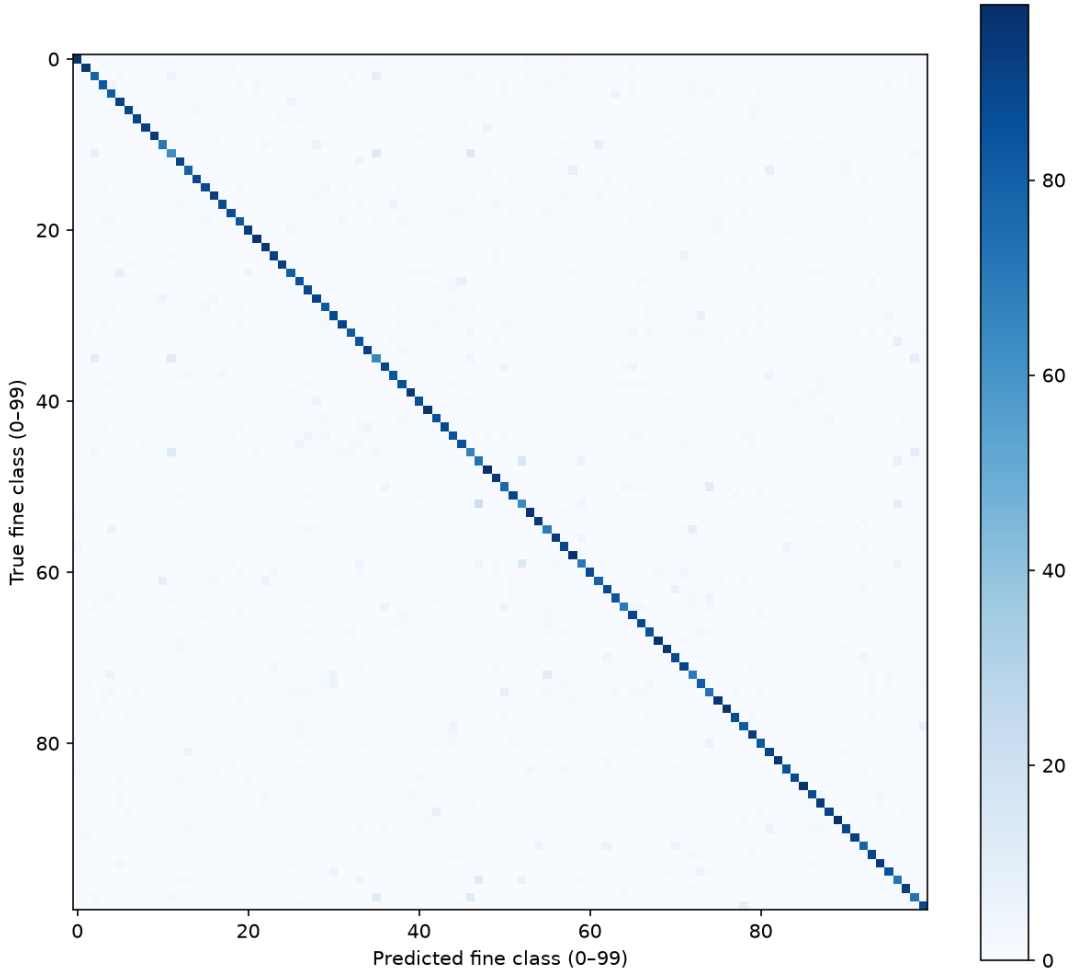
Hình 4.3: ResNet18 pretrained full: đường cong ở seed 42.



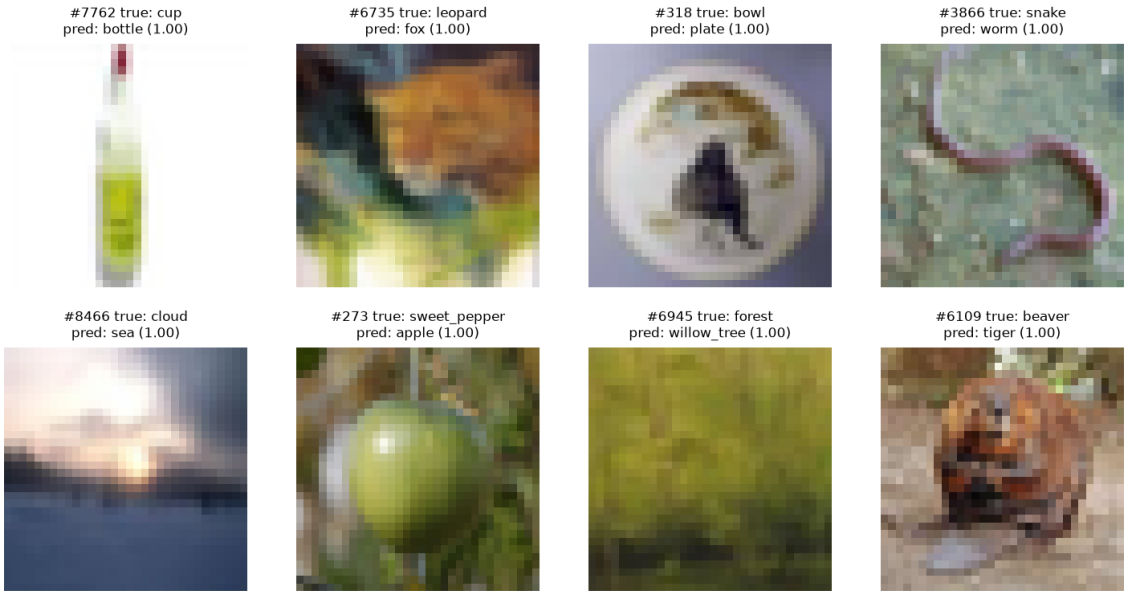
Hình 4.4: Swin-T pretrained full: đường cong ở seed 42.



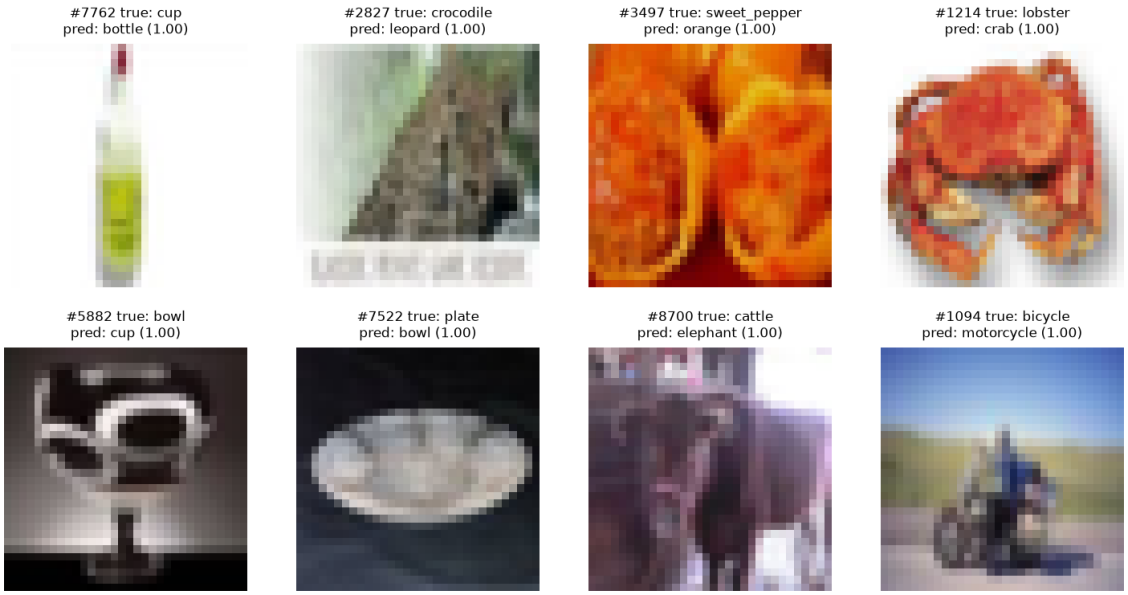
Hình 4.5: Confusion matrix test của ResNet18 full ở seed 42; hàng là nhãn thực tế.



Hình 4.6: Confusion matrix test của Swin-T full ở seed 42.



Hình 4.7: Các dự đoán sai có confidence cao của ResNet18 full ở seed 42, hiển thị ảnh gốc 32×32.



Hình 4.8: Các dự đoán sai có confidence cao của Swin-T full ở seed 42.

Chương 5

Kết luận và giới hạn

5.1 Kết luận

Trong thiết lập CIFAR-100 đầy đủ với 20 epoch, Swin-T fine-tuning toàn bộ cho test accuracy cao nhất, đạt $86.63 \pm 0.22\%$ và macro-F1 0.8663. Fine-tuning stage cuối hoặc toàn bộ cải thiện kết quả so với chỉ học head ở cả hai kiến trúc. ResNet18 scratch học được biểu diễn hữu ích trong ngân sách này, còn Swin-T scratch chưa học tốt. Lợi thế accuracy của Swin-T full đi kèm thời gian và bộ nhớ cao hơn ResNet18 full.

5.2 Tổ chức mã nguồn và tái lập

Mã được chuyển sang `src/image/`, cấu hình ở `configs/image.json`.

Notebook `notebooks/image.ipynb` chạy lại kiểm tra dữ liệu và phân tích prediction bằng CPU.

`python -m image.analysis` kiểm tra provenance và tính lại chỉ số. Training mới dùng `image.run_suite`; chữ ký nguồn mới không resume checkpoint của CO5085.

`python scripts/build_report.py` biên dịch LaTeX thành PDF. Các lệnh đầy đủ được ghi trong `docs/IMAGE_REUSE.md`.

Các kiểm thử không tải dataset hoặc trọng số pretrained. Phạm vi kiểm tra gồm chia tập không chồng lấp, accuracy/F1, đóng băng tham số và thống kê BatchNorm, stochastic depth, augmentation theo epoch và worker, khôi phục checkpoint, RNG, optimizer, scheduler, kiểm tra signature và xử lý checkpoint bị gián đoạn. Kiểm tra bổ sung của Image đối chiếu bằng chứng nhập và tính lại chỉ số từ predictions.

5.3 Giới hạn

Ảnh gốc 32×32 và đầu vào 96×96 khác với kích thước phổ biến 224×224 của pretrained. Thực nghiệm chỉ khảo sát CIFAR-100, hai kiến trúc và ngân sách 20 epoch; đặc biệt, scratch có thể tiếp tục cải thiện nếu được huấn luyện lâu hơn. Hai họ còn khác capacity và recipe ImageNet. Ba seed mô tả biến thiên giữa các lượt khởi tạo, với cùng phần cứng và thiết lập workers. Tình trạng license của dữ liệu vẫn là chưa rõ.

Báo cáo sử dụng AI được đính kèm tại tệp `AI_USE_REPORT.md` trong repository.

Tài liệu tham khảo

[1] Lê Thành Sách. *Course Assignment: Nền tảng lập trình cho phân tích và trực quan dữ liệu*. CO5177, v4.0, 14/09/2026. Nội dung yêu cầu hiện có trong trang requirements của repository.

[2] A. Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. 2009. Trang dataset: `https://www.cs.toronto.edu/~kriz/cifar.html`.

[3] uoft-cs. *CIFAR100 dataset card*; license: unknown. `https://huggingface.co/datasets/uoft-cs/cifar100`.

[4] K. He và cộng sự. *Deep Residual Learning for Image Recognition*. CVPR2016. `https://arxiv.org/abs/1512.03385`.

[5] Z. Liu và cộng sự. *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. ICCV2021. `https://arxiv.org/abs/2103.14030`.

[6] Torchvision. ResNet18 và Swin-T model API; sử dụng weight enum IMAGENET1K_V1. `https://docs.pytorch.org/vision/stable/models.html`.